桃園即時路況通設計文件

B1229038廖奕翰
B1229051黃一哲
B1229044李昊軒
B1229016賈子樂

指導教授:李季青教授

1. 系統簡介

1.1 規格目的

本系統《桃園路況即時通》之開發目的，旨在建立一個高效、精準且具備公民參與特性的智慧交通資訊平台。桃園市作為北台灣重要的交通樞紐，其路況資訊的即時性對於通勤族與物流業者至關重要。然而，現有官方交通 API 雖提供量化數據（如平均車速），但在突發事件（如輕微事故、暫時施工）的捕捉上往往存在時滯；而傳統廣播資訊則受限於單向傳播，缺乏空間定位的精確度。

本規格書詳述之系統，核心目的在於透過「多源資料融合（Multi-Source Data Fusion) 技術，將官方感測器數據與警廣（PBS）質化資訊進行空間對齊。此外，本系統特別強調「公民共創」之理念，開發易用之語音回報機制，使每位駕駛人皆能成為即時路況的感測節點。本文件不僅定義系統之功能架構，亦作為後續開發「紅燈誤判排除機制」與「語音回報系統（Speech-to-Text）」之核心技術規範，確保系統能產出高信賴度的壅塞分數，優化市民的出行決策。

1.2 規格範圍

本系統之規格設計涵蓋從後端數據採集到前端視覺化應用的完整生命週期，具體範圍包含以下三大範疇

資料處理與空間運算層：

i.

多源數據擷取：自動化擷取桃園市政府交通局之即時流速 API 以及內政部警政署廣播電臺（PBS）之即時路況事件 JSON。

ii.

空間資料庫建模：利用 PostgreSQL 結合 PostGIS 模組，建立桃園全區道路幾何模型（LineString）與事件點位（Point），實現路網與事件的精確時空對齊。

iii.

群眾回報數據處理：標準化使用者端上傳之位置、方向與事件類型數據。

核心分析與演算法層：

i.

壅塞分析演算法：開發加權評分模型，結合路段即時速率與周邊事件密度（300公尺半徑內之事故與施工）產出動態壅塞分數。

ii.

紅燈誤判排除（核心亮點）：透過使用者前進方位（Bearing）與靜止時間之邏輯判斷，排除因路口紅燈停等造成的流速異常誤判。

iii.

替代路線建議邏輯：基於 Dijkstra 最短路徑演算法，當偵測到目標路段為嚴重壅塞（紅色）時，主動運算時間成本最低之避讓路徑。

應用服務與介面層：

i.

Android 行動端應用：基於 Jetpack Compose 與 Google Maps SDK 實作之地圖視覺化介面，提供三色燈號路況圖層。

ii.

語音互動介面：利用 Speech-to-Text (STT) 技術實現免動手路況回報，並整合地理編碼（Reverse Geocoding）自動帶入所在路名。

iii.

後端管理介面：提供後端 API 供前端呼叫，並預留資料視覺化儀表板供市政分析使用。

1.3 參考文件

本系統之設計參考並遵循以下相關標準與技術規範：

1.政府公開資訊規格：桃園市政府交通局－即時交通資訊（ATIS）公開資料格式規範、內政部警政署廣播電臺路況資訊數據交換規範。

2.地理空間標準：Open Geospatial Consortium (OGC) 空間資料結構、PostGIS 空間查詢語法規範。

3.地圖開發規範：Google Maps Platform - Maps SDK for Android API 參考指南。

4.系統架構參考：Dijkstra 演算法於有向路網之應用、微服務架構與 RESTful API 設計原則。

5.專案開發基礎：長庚大學《桃園路況即時通》專題提案計畫書。

2. 系統概述

2.1 系統目標

本系統《桃園路況即時通》之核心開發目標，在於建構一個融合官方數據與民眾協作的智慧交通生態系，具體目標涵蓋以下四大面向：

- 解決交通壅塞與提升行車效率：針對桃園市日益嚴重的交通壅塞問題，提供用路人精準的避讓策略，達成「減少塞車、節省時間、提升行車效率」之目標，協助市民大幅降低通勤延誤之時間成本。
- 提供即時且在地化之路況預警：透過多源資料融合技術，達成極致的『即時性』與『在地性』，提供精細至單一路段的動態壅塞預警，有效彌補傳統警廣在時效性與地域精準度上之不足。
- 推動公民共創與智慧交通治理：結合易用的免持語音回報機制，推動桃園邁向智慧交通與公民共創治理之全新典範。
- 建立智慧決策支援與數據治理：藉由長期蒐集、儲存並分析車流與事件數據，探討都市建設、道路施工與大型活動對周邊交通之實質影響，提供市政單位作為未來交通決策之客觀大數據依據。

2.2 系統範圍

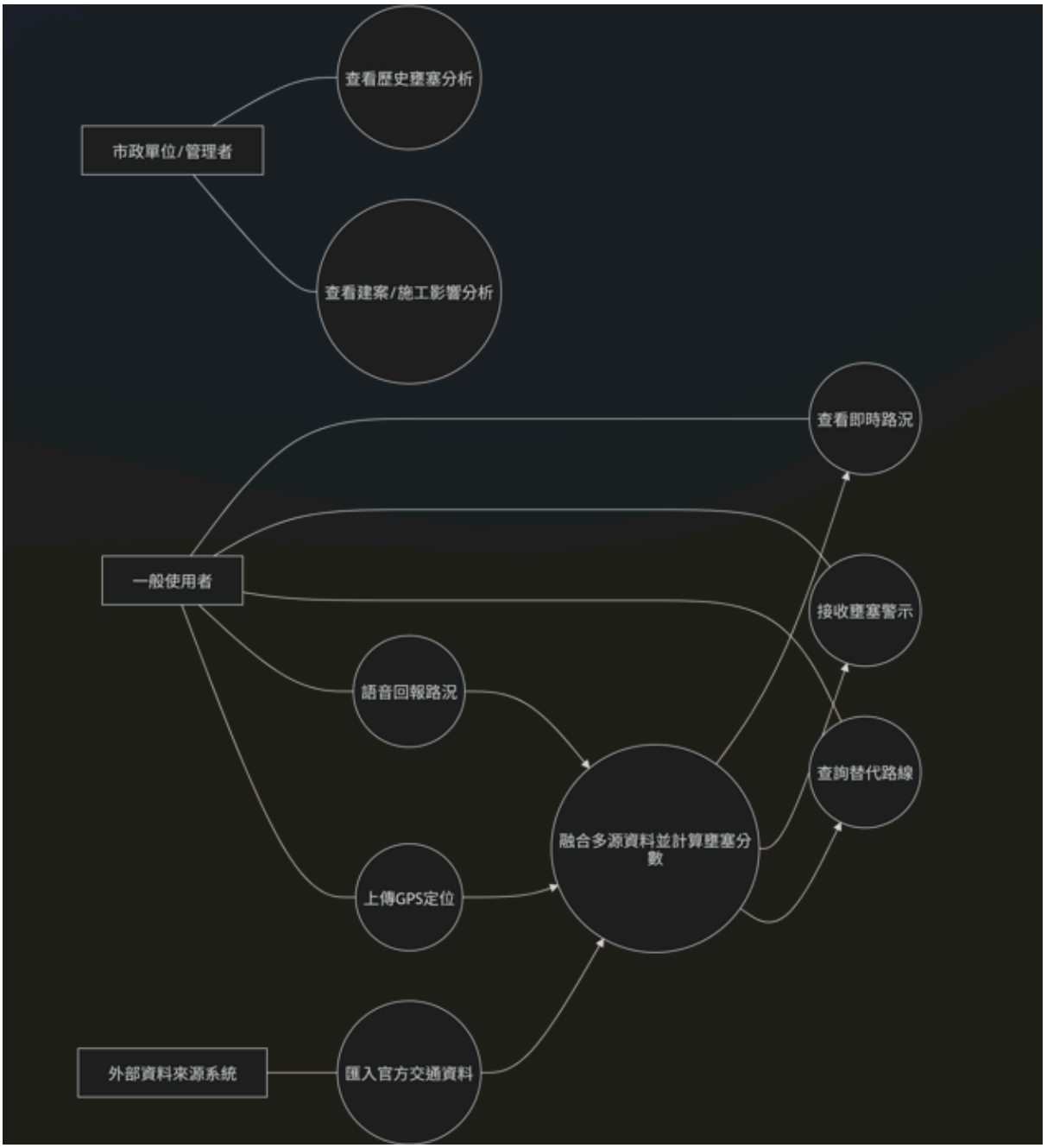
服務範圍為桃園市道路網，包含：

本系統之服務範圍涵蓋空間、資料及應用層面，具體定義如下：

- 地理與路網範圍：本系統之空間範圍專屬於「桃園市」之交通道路網，涵蓋市內主要幹道、次要幹道及易壅塞之支線道路。
- 資料整合範圍：系統之數據基底匯聚三大來源，包含桃園市政府交通局 API 之即時感測流速、內政部警政署廣播電臺（PBS）之即時路況事件，以及由本系統使用者端上傳之群眾回報資料。

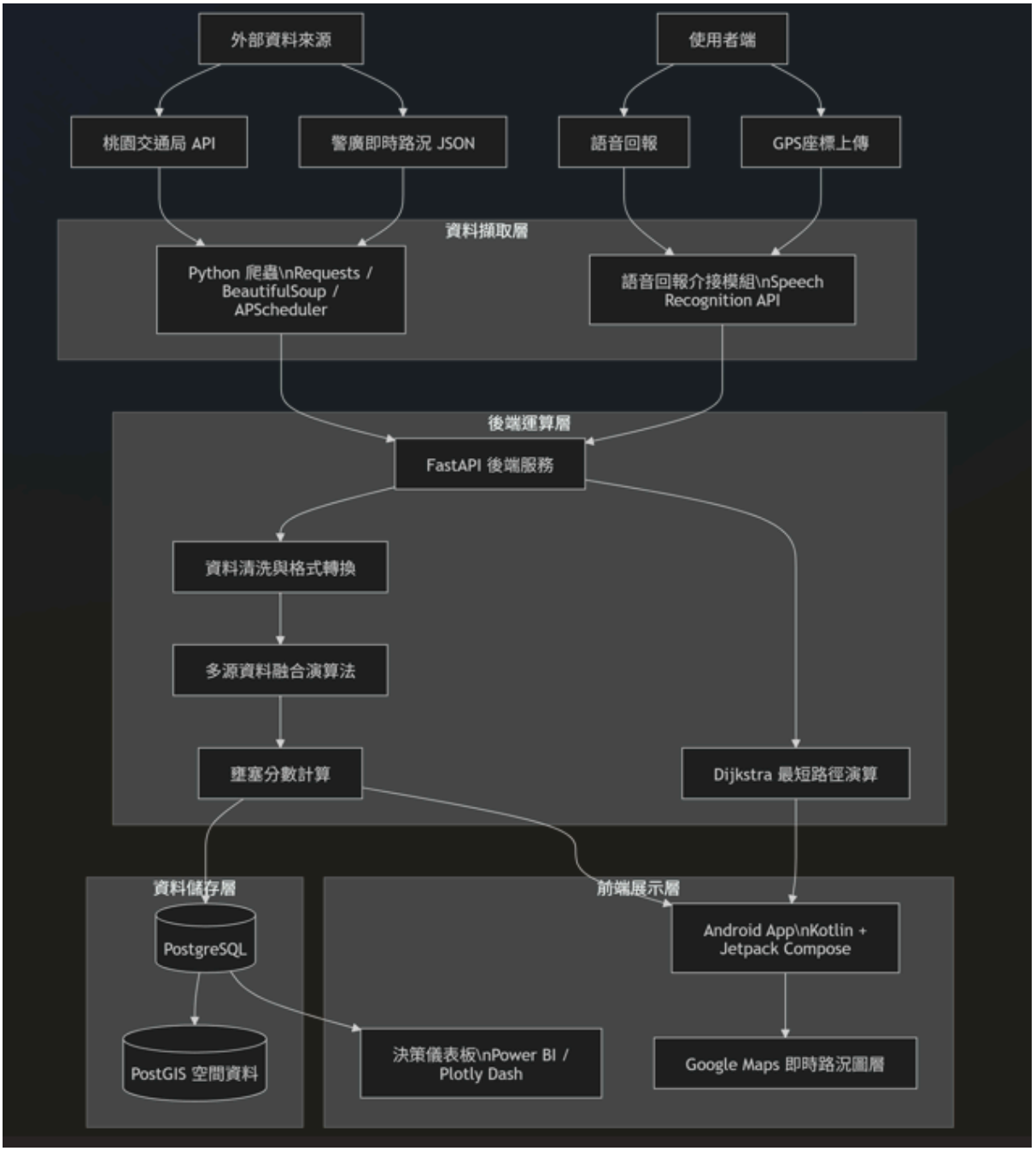
核心應用與功能範圍：透過**「官方數據＋民眾眾包」**之互補雙引擎模式，系統提供包含：三色燈號即時路況視覺化、動態壅塞警示、基於最短路徑演算之替代路線建議、語音

回報系統，以及針對壅塞熱點與建案/施工關聯性之深度分析功能。



(使用案例圖)

2.3 系統架構



(系統架構圖)

本系統採用微服務概念設計之三層式架構(Three-tier Architecture)，以確保系統之高可用性與擴展性，各層級架構詳述如下：

- 資料擷取層 ：

  i. 自動化資料爬蟲：透過 Python 開發工具(如 Beautiful Soup、Requests、APScheduler)，定時且自動化擷取桃園交通 API 及警廣 JSON 格式資料。

  ii. 語音回報介接：負責接收並處理前端使用者透過 Speech Recognition API 所轉換之語音回報文字與座標，作為即時路況之輔助校正節點。

- 後端運算層：

  i. **API** 與運算核心：採用 Python 結合 FastAPI 框架建置，負責高效能的資料清洗、處理與即時狀態產製。

  ii. 空間資料庫建模：導入 PostgreSQL 關聯式資料庫，並搭載 PostGIS 空間擴充模組，以儲存、索引並計算複雜的地理空間幾何資料。

  iii. 排程與融合演算：伺服器端配置每 5 分鐘執行一次之排程運算，執行多源資料融合演算法，動態計算各路段之「壅塞分數」。

  iv. 動態路徑運算：負責實作 Dijkstra 等最短路徑演算法，於偵測到前方路段為嚴重壅塞時，即時為使用者提供最佳替代繞路建議。

- 前端展示層：

  i. 行動端應用：基於 Google Maps Platform 的 Maps SDK for Android 進行原生開發，透過直覺的紅、黃、綠三色燈號，於地圖介面上即時繪製道路狀態圖層。

  ii. 決策儀表板：結合 Power BI 或 Plotly Dash 等視覺化工具，開發互動式交通決策儀表板，提供市政單位進行歷史趨勢分析。

2.4 軟硬體需求

為支撐上述系統架構之高效運行，本系統建構之軟硬體技術堆疊(Tech Stack)需求如下：

- 軟體建構項目：

  i. 後端開發框架：Python 搭配 FastAPI，以支援高併發之 RESTful API 服務。

  ii. 資料庫系統：PostgreSQL(必須包含 PostGIS 空間擴充套件)。

iii. 前端開發平台：Android 原生開發環境（Kotlin + Jetpack Compose），並深度整合 Google Maps Platform API。

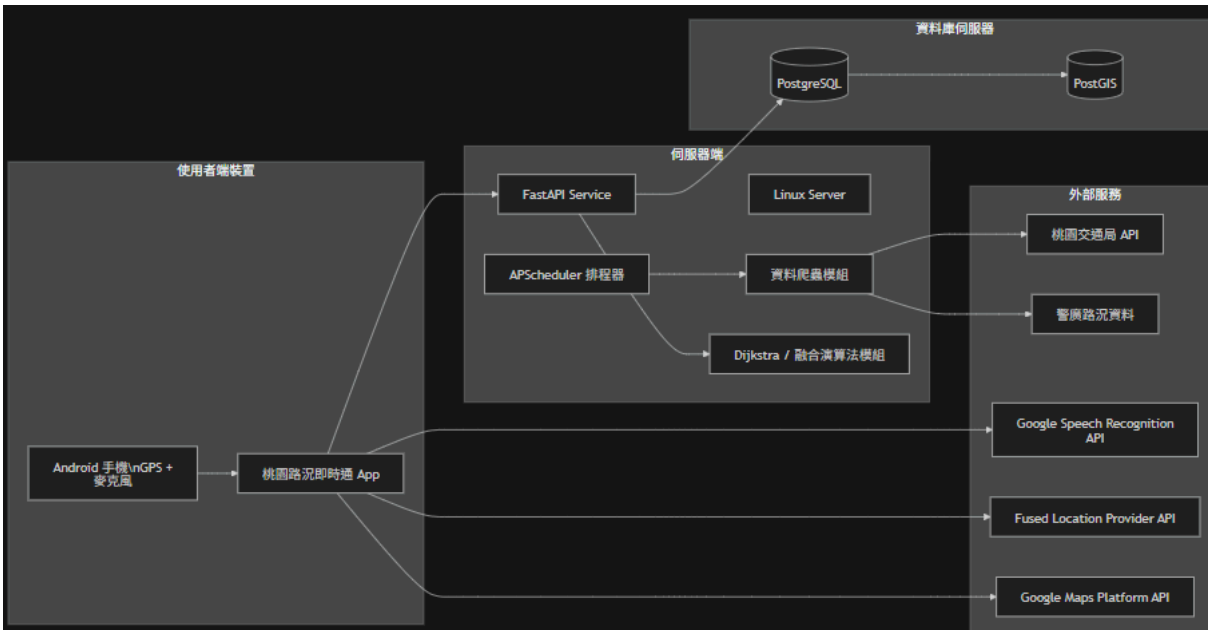
iv. 資料分析與視覺化：Power BI 或 Plotly Dash。

v. 第三方 **API** 服務：Google Speech Recognition API（語音轉文字）與 Fused Location Provider API（高精度定位）。

- 硬體需求概述：

i. 伺服器端：需部署於具備高效能 CPU 與充足記憶體之雲端或實體伺服器（建議規格如：16GB RAM / 8 Cores / SSD），以從容應付後端排程資料爬取、龐大空間幾何資料（LineString/Point）運算及常態性路徑演算。

ii.Android 手機（具 GPS 與麥克風）



(系統佈署圖)

2.5 系統環境

系統營運與開發所依賴之軟硬體環境配置規範如下：

- 伺服器部署環境：後端需部署於支援 Python 執行環境及穩定運行 PostgreSQL/PostGIS 服務之 Linux 作業系統環境。
- 前端使用者環境：

i. 作業系統：支援 Android 8.0 (API Level 26) 及以上版本之行動裝置。

ii. 硬體感測要求：裝置必須具備運作正常之 GPS 晶片與麥克風，以確保系統能精確獲取座標並執行免動手語音回報功能。

iii. 網路連線：需處於穩定的 4G/5G 或 Wi-Fi 網際網路連線狀態，以接收即時路況圖層之串流更新。

- 開發與測試環境：開發團隊需配置完整之 Python 虛擬環境、Android Studio、資料庫視覺化軟體（如 DBeaver 或 pgAdmin）以及 API 開發測試工具。

2.6 系統限制

本系統在實際運作與技術實作上，存在以下客觀限制與挑戰：

- **API** 更新頻率與時滯：官方交通局與警廣資料之更新頻率受限於來源端系統之排程設定，本系統雖設定為每 5 分鐘同步一次，但在極短時間內發生的突發狀況仍可能有短暫之資訊時滯。
- **GPS** 訊號與定位誤差：前端執行「紅燈誤判排除機制」高度依賴使用者手機的 GPS 與陀螺儀。在隧道、高架橋下、高樓密集區或天候不佳時，可能產生定位飄移或前進方位之判斷誤差。
- 群眾回報數據之可信度：由於系統開放使用者即時回報路況，回報資訊受限於駕駛人的主觀判斷。初期回報數據之可信度可能不穩定，需仰賴後續演算法過濾惡意或偏誤之回報節點。
- 網路延遲影響即時性：即時地圖繪製與路徑演算依賴穩定的網路連線，若使用者處於網路死角或行動裝置運算資源不足，可能導致圖層渲染延遲或錯失即時避讓時機。

---

3. 設計內容

3.1 軟/硬體建構項目架構

本系統採前後端分離架構，結合空間資料庫與即時資料流處理，具體架構層次如下：

- 資料擷取層（Data Ingestion Layer）：包含自動化爬蟲（Crawler），負責定時調用外部 API 獲取異質資料。

- 資料儲存與計算層（Storage & Computation Layer）：以 PostGIS 為核心，不僅儲存屬性資料，更處理路段的幾何空間運算（Spatial Join）。
- 核心邏輯層（Business Logic Layer）：包含壅塞演算法與紅燈誤判過濾機制，將原始數據轉化為路況特徵。
- 介面服務層（API Service Layer）：利用 FastAPI 提供高性能的 RESTful API，封裝成 JSON 格式供移動端調用。
- 客戶端展示層（Client Presentation Layer）：以 Android (Jetpack Compose) 為主體，整合 Google Maps SDK 提供直觀的視覺化介面與使用者回報功能。

3.2 軟硬體組件設計說明

（1）資料擷取模組（Crawler）

- 高頻抓取機制：針對桃園交通局路段車速 API，設定每 5 秒執行一次異步請求，確保路況的即時性。•異質資料解析：透過解析警廣（Police Radio）提供的非結構化或 JSON 格式事件資料，提取關鍵字（如「車多」、「事故」）。•異步寫入：採用非阻塞式 I/O 將資料定時寫入 PostgreSQL，避免大量寫入操作鎖定資料表。

（2）資料庫設計（PostGIS）

- `traffic_speed_raw`：紀錄路段 ID、即時時速、時間戳
- `roadwork_raw`：紀錄施工、事故等突發事件
- `user_report`：儲存使用者透過 App 回傳的即時座標與回報類別（語音/點選）
- `road_segment`：定義路段的幾何路網（LineString），用於空間緩衝區比對

（3）壅塞分數演算法

- 動態權重模型：系統根據路段特性計算動態壅塞分數（Congestion Score），未來優化公式如下：Score = (0.5 \times SpeedScore) + (0.2 \times EventDensity) + (0.2 \times UserFeedback) + (0.1 \times Duration)
- 參數說明：其中 SpeedScore 採標準偏差法，計算當前車速偏離該路段平均車速的程度。

（4）紅燈誤判排除機制

為解決車輛因停等紅燈導致的「偽壅塞」現象，設計以下邏輯：

- 時間窗口過濾：低速狀態必須持續超過 6 分鐘（涵蓋典型長紅燈週期）。
- 多源驗證：結合事件 API 或 2 筆以上的使用者回報進行二次確認。若無額外證據，則維持「緩行」而非「壅塞」。

（5）Android 前端建構（Kotlin/Compose）

- 地圖引擎：使用 Google Maps SDK 搭配 maps-compose 庫，實現高效能的地圖渲染。
- 渲染邏輯：根據 API 回傳的 congestion_score 渲染 Polyline 顏色（Green: >60km/h, Yellow: 30-60km/h, Red: <30km/h）。
- 生命週期管理：利用 androidx.lifecycle 監聽 App 狀態，實現自動定時更新（Polling）機制。

3.3 介面設計說明

- （1）地圖主畫面 **(Main Map View)**
  - 圖層疊加：整合 Google Maps 基礎底圖、路況熱力圖層、以及事件標記圖層（Marker）。
  - 聚合顯示：當標記過多時，自動啟用 Cluster 渲染，避免介面混亂。
- （2）資訊視窗 **(Bottom Sheet)**
  - 彈出風格：當使用者點擊特定路段或事件標記時，由下方彈出 Material Design 3 風格的 Bottom Sheet。
  - 顯示內容：
    - 具體路段名稱（如「中正路-復興路口」）。
    - 路況詳情（顯示目前平均時速及資料來源）。
    - 互動按鈕（提供「導航至此」或「回報異常」之功能）。
- （3）回報介面 **(Report UI)**
  - 多元輸入：支援傳統點選選單與語音輸入（Speech-to-Text 整合預留）。
  - 自動定位：透過 play-services-location 自動帶入 GPS 座標，減少使用者輸入成

本。

3.4 作業程序設計說明

系統運行的標準化流程如下：

1. 後端循環：爬蟲抓取 $\rightarrow$ 空間資料清理（Cleaning） $\rightarrow$ 存入 PostGIS。
2. 邏輯處理：演算法定時掃描資料表 $\rightarrow$ 更新 congestion_score。
3. API 調配：FastAPI 監聽客戶端請求 $\rightarrow$ 執行空間查詢 $\rightarrow$ 封裝 JSON 回傳。
4. 客戶端展示：Android App 獲取資料 $\rightarrow$ 動態繪製 Polyline $\rightarrow$ 視覺化呈現給使用者。

3.5 輸入/輸出設計說明

- 輸入項 **(Input)**：
  - 公部門開放資料（車速、施工、事件）。
  - 使用者主動回報資料（座標、類別、語音）。
  - 端點硬體感測（GPS 經緯度、羅盤方向、移動速度）。
- 輸出項 **(Output)**：
  - 視覺化路況地圖（綠/黃/紅顏色編碼）。
  - 即時語音警示（未來功能）。
  - 管理端 Dashboard 統計報表（平均擁堵時段、高發事故區域）。

3.6 其他設計說明

- 空間效能優化：利用 PostGIS 的 ST_DWithin 進行快速空間比對，確保 API 在高併發下仍能快速回應。
- 網路通訊：使用 Retrofit 2 搭配 OkHttp 處理 RESTful 請求，實作攔截器（Interceptor）進行 Log 監控。
- 擴充性預留：
  - 語音辨識：預計導入 Google Cloud Speech API。
  - 導航引擎：預計導入 Dijkstra 演算法結合即時路權進行路徑優化。

4. 需求追溯與版本管理

功能	狀態
多源資料擷取	已完成
路況顯示	已完成
壅塞分數	開發中
紅燈誤判	開發中
回報系統	開發中
儀表板	未完成

附錄

API 範例


```
{
 "road_id": "TY001",
 "speed": 20,
 "congestion_score": 75,
 "status": "jam",
 "color": "red"
}
```

檔案對照表

模組	檔案
前端	MapScreen.kt
後端	main.py

爬蟲	speed_crawler.py
演算法	congestion.py
資料庫	PostgreSQL

報表

報表	說明
壅塞統計	各路段壅塞比例
熱點圖	塞車分布
congestion_hotspot	前十大壅塞路段
user_report	使用者活躍分析